



Rust Seekho — Masti Mein, DSA Ke Saath

Kiske liye hai ye doc? Jo banda Rust zero se seekhna chahta hai aur phir LeetCode / Codeforces pe Rust mein DSA solve karna chahta hai — bina har second syntax pe atke.

Kaise padhna hai? Part 1 (Basics) seedha padho. Part 2 (Ownership) ko dhyan se padho — Rust ka asli "boss level" wahi hai. Part 3–4 (Collections + DSA patterns) ko **cheat sheet** ki tarah use karo — problem solve karte waqt khol ke rakho.

Golden rule: Compiler tumhara dushman nahi, tumhara strict sir hai. Gussa karta hai, par error message mein solution bhi likh deta hai. **Error message poora padho.** 🙏



Table of Contents

- **Part 0 — Setup**
 - 0. Rust install aur pehla program
- **Part 1 — Basics (Syntax ka darr khatam)**
 - 1. Variables: `let`, `mut`, shadowing
 - 2. Data types — kaunsa kab use karna hai
 - 3. Functions aur "expression" ka jaadu
 - 4. Control flow: `if`, `loop`, `while`, `for`
- **Part 2 — Ownership (Boss Level 🧠)**
 - 5. Ownership — Rust ka dil
 - 6. Borrowing: `&` aur `&mut`
- **Part 3 — Real Rust**
 - 7. Strings — sabse zyada confuse karne wali cheez
 - 8. Vec — tumhara best friend
 - 9. Iterators — Rust ka superpower
 - 10. Option aur Result — null ka safe version
 - 11. `match` — switch-case ka baap
 - 12. Struct, impl, enum, trait
 - 13. Closures — chhote anonymous functions
 - 14. Collections: HashMap, HashSet, BTreeMap, VecDeque, BinaryHeap
- **Part 4 — DSA Mode ON 🔥**
 - 15. LeetCode aur Codeforces template
 - 16. DSA patterns cookbook
 - 17. Linked List aur Tree (LeetCode wala dard)
 - 18. Common errors aur unka ilaaj
 - 19. C++/Python → Rust cheat sheet
 - 20. Practice roadmap

Part 0 — Setup

0. Rust install aur pehla program

```
# Install (Mac/Linux)
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh

# Check
rustc --version
cargo --version
```

Cargo = Rust ka npm/pip. Project banata hai, build karta hai, run karta hai.

```
cargo new dsa_practice    # naya project
cd dsa_practice
cargo run                 # build + run
```

src/main.rs kuch aisa dikhega:

```
fn main() {
    println!("Namaste Rust! 🦀");
}
```

- `fn main()` → program yahin se start hota hai.
- `println!` → print karta hai. Ye `!` dekh rahe ho? Iska matlab ye **macro** hai, function nahi. Abhi ke liye bas itna yaad rakho: `!` wali cheezein "special functions" hain.

Print karne ke tareeke:

```
let naam = "Sanju";
let age = 25;
println!("Mera naam {} hai, age {}", naam, age);
println!("Mera naam {naam} hai, age {age}"); // direct variable andar daal sakte ho
println!("{:?}", vec![1, 2, 3]);             // {:?} = debug print (Vec, tuple, etc. ke liye)
println!("{:.2}", 3.14159);                  // 3.14 (2 decimal)
```

💡 **Tip:** Kuch bhi print nahi ho raha `{}` se? `{:?}` try karo. Vec, HashMap, Option — sab `{:?}` se print hote hain.

Quick try karna hai bina install ke? → play.rust-lang.org 🎮

Part 1 — Basics

1. Variables: `let`, `mut`, shadowing

Rust mein variable **by default immutable** hota hai. Matlab ek baar value di, toh change nahi kar sakte. Shaadi jaisa commitment. 

```
let x = 5;
x = 6;      // ❌ ERROR: cannot assign twice to immutable variable
```

Change karna hai? `mut` lagao:

```
let mut x = 5;
x = 6;      // ✅ chalega
x += 1;     // ✅ 7
```

⚠️ Rust mein `x++` aur `x--` **nahi hota**. `x += 1` likho.

Type khud likhna (optional)

Rust smart hai, type khud guess kar leta hai. Par tum bata bhi sakte ho:

```
let a = 10;           // Rust samjha: i32
let b: i64 = 10;      // tumne bola: i64
let c = 10i64;        // ye bhi i64 (suffix style)
let d = 1_000_000;    // underscore sirf readability ke liye, = 1000000
```

Shadowing — same naam, naya variable

```
let x = "42";         // x ek string hai
let x: i32 = x.parse().unwrap(); // ab x ek number hai (naya variable, purana chhup gaya)
```

Ye DSA mein bahut kaam aata hai — input string ko number banana, `Vec` ko sort karke same naam rakhna, etc.

Constants

```
const MOD: i64 = 1_000_000_007; // type likhna ZAROORI hai const mein
const INF: i64 = i64::MAX;
```

2. Data types — kaunsa kab use karna hai

Integers

Type	Range	Kab use karo
<code>i32</code>	$\sim \pm 2.1 \times 10^9$	LeetCode ka default (function signatures mein yahi hota hai)
<code>i64</code>	$\sim \pm 9.2 \times 10^{18}$	Sum, product, jab bhi overflow ka darr ho
<code>usize</code>	0 se upar (64-bit system pe u64 jitna)	Index, length, size — <code>v[i]</code> mein <code>i</code> hamesha <code>usize</code>
<code>u64</code>	0 to 1.8×10^{19}	Hashing, bitmask
<code>i128</code>	bahut bada	Rare cases, bade multiplication

DSA ka golden rule:

- **Index ke liye** → `usize`
- **Values/answer ke liye** → `i64` (safe side)
- LeetCode ne `i32` diya hai → wahi return karo, andar `i64` mein calculate karo

```
let max = i32::MAX;    // 2147483647
let min = i64::MIN;
let big = usize::MAX;
```

Floats, bool, char

```
let pi: f64 = 3.14;    // hamesha f64 use karo, f32 nahi
let ok: bool = true;
let c: char = 'A';     // single quote = char
let s: &str = "hello"; // double quote = string
```

Type casting — `as` keyword

Rust **automatic conversion nahi karta**. `i32 + i64` direct nahi jodega. Khud bolna padega:

```
let a: i32 = 5;
let b: i64 = 10;
let c = a as i64 + b;           // ✅

let n: usize = v.len();
let half = n as i32 / 2;        // usize → i32

let ch = 'c';
let idx = (ch as u8 - b'a') as usize; // 'c' → 2 (alphabet index — DSA mein bahut common!)
let back = (b'a' + 2) as char;      // 2 → 'c'
```

🔥 **DSA tip:** `b'a'` ek byte (u8) hai jiska value 97 hai. `'a'` ek char hai. Bytes ke saath math karna aasaan hai.

Overflow — Rust ka strictness

Debug mode mein agar `i32` overflow hua toh program **panic** (crash) kar jata hai. C++ ki tarah chupchaap galat answer nahi deta. Ye actually achha hai!

```
let x: i32 = i32::MAX;
let y = x + 1;           // 🚨 panic: attempt to add with overflow (debug mode)
let y = x as i64 + 1;    // ✅
let y = x.wrapping_add(1); // jaan-boojh ke wrap karna ho toh
let y = x.checked_add(1); // Option<i32> deta hai: overflow pe None
```

Tuples

```
let p = (3, 4);
let (x, y) = p;           // destructuring
println!("{}", p.0, p.1);

let pair: (i32, String) = (1, String::from("hi"));
```

Arrays (fixed size)

```
let arr = [0; 26];           // 26 zeros — frequency count ke liye perfect
let arr2 = [1, 2, 3];
let mut freq = [0; 26];
freq[(b'z' - b'a') as usize] += 1;
```

Size compile time pe pata hona chahiye. Runtime size chahiye → `Vec` use karo (section 8).

3. Functions aur "expression" ka jaadu

```
fn add(a: i32, b: i32) -> i32 {
    a + b           // ⚠️ NO semicolon = ye value return hogi
}

fn main() {
    let s = add(2, 3);
    println!("{}", s);
}
```

Sabse important baat: Rust mein last line pe semicolon nahi lagaya → wo value return ho jaati hai. Semicolon lagaya → kuch return nahi hota (actually `()` return hota hai, jo "kuch nahi" hai).

```
fn square(x: i32) -> i32 {
    x * x          // ✅ returns x*x
}

fn square_galat(x: i32) -> i32 {
    x * x;         // ❌ ERROR: expected i32, found ()
}
```

return keyword bhi hai — **early return** ke liye use karo:

```
fn find(v: &Vec<i32>, target: i32) -> i32 {
    for i in 0..v.len() {
        if v[i] == target {
            return i as i32; // beech mein nikalna ho toh return
        }
    }
    -1                        // last value, bina return ke
}
```

Blocks bhi value dete hain

```
let y = {
    let a = 3;
    a * 2      // y = 6
};
```

4. Control flow: if, loop, while, for

if — ye bhi value deta hai!

```
let x = 7;
if x % 2 == 0 {
    println!("even");
} else if x % 3 == 0 {
    println!("div by 3");
} else {
    println!("kuch aur");
}

// ternary operator ki jagah:
let parity = if x % 2 == 0 { "even" } else { "odd" };
let mx = if a > b { a } else { b }; // ya simply a.max(b)
```

⚠️ Condition ke around bracket `()` nahi chahiye, aur condition **bool hi honi chahiye**. `if x { }` jahan x integer hai — nahi chalega. `if x != 0 { }` likho.

for loop — ranges ke saath

```
for i in 0..5 { }           // 0,1,2,3,4 (5 excluded)
for i in 0..=5 { }          // 0,1,2,3,4,5 (5 included)
for i in (0..5).rev() { }   // 4,3,2,1,0 (ultra)
for i in (0..10).step_by(2) { } // 0,2,4,6,8

let v = vec![10, 20, 30];
for x in &v { }              // har element (reference)
for (i, x) in v.iter().enumerate() { } // index + value dono
for x in &mut v { *x += 1; } // har element modify karo (v must be mut)
```



DSA must-know: C++ ka `for (int i = n-1; i >= 0; i--)` → Rust mein `for i in (0..n).rev()`. Ye bahut use hoga.

while

```
let mut l = 0;
let mut r = v.len() - 1;
while l < r {
    l += 1;
    r -= 1;
}
```

loop — infinite loop (break se bahar aao)

```
let mut count = 0;
let result = loop {
    count += 1;
    if count == 10 {
        break count * 2; // loop bhi value return kar sakta hai!
    }
};
```

Labeled break — nested loop se ek dum bahar

```
'outer: for i in 0..n {
    for j in 0..m {
        if grid[i][j] == 1 {
            break 'outer; // dono loops se bahar
        }
    }
}
```

Part 2 — Ownership (Boss Level 🧠)

Ye wo part hai jahan log Rust chhod dete hain. Tum nahi chhodoge. Analogy se samjhenge, bilkul aasan hai.

5. Ownership — Rust ka dil

Analogy: Socho tumhare paas ek **book** hai (data). Rust ke 3 rules:

1. Har book ka **ek hi malik** (owner) hota hai.
2. Ek time pe **sirf ek malik**.
3. Malik scope se bahar gaya (function khatam, `}` aa gaya) → book **destroy**. (Memory free — garbage collector ki zaroorat hi nahi!)

Move — book de di, ab tumhari nahi rahi

```
let a = String::from("hello");
let b = a;           // book 'a' ne 'b' ko de di (MOVE)
println!("{}", a);   // ❌ ERROR: borrow of moved value: `a`
```

`a` ab khaali haath hai. Function mein pass karne pe bhi yahi hota hai:

```
fn print_it(s: String) { println!("{}", s); }

let s = String::from("hi");
print_it(s);    // s move ho gaya function mein
print_it(s);    // ❌ ERROR: use of moved value
```

Clone — photocopy kara lo

```
let a = String::from("hello");
let b = a.clone();    // nayi copy bani
println!("{}", a, b); // ✅ dono valid
```

Clone **costly** ho sakta hai (poora data copy hota hai). DSA mein `vec.clone()` loop ke andar mat karo — TLE aa jayega.

Copy types — inka koi jhanjhat nahi

Chhote, simple types (`i32`, `i64`, `usize`, `f64`, `bool`, `char`, aur inke tuples) **automatically copy** hote hain. Move ka chakkar nahi:

```
let x = 5;
let y = x;           // copy hua
println!("{}", x, y); // ✅ dono valid
```

Yaad rakho: Numbers = tension free. `String`, `Vec`, `HashMap` = ownership ka dhyan rakho.

6. Borrowing: `&` aur `&mut`

Har baar book dena (move) ya photocopy (clone) karna practical nahi. Toh **udhaar do** (borrow)!

`&` — Padhne ke liye udhaar (immutable borrow)

```
fn length(s: &String) -> usize { // & = "main sirf padhunga, malik tum hi raho"
    s.len()
}

let s = String::from("hello");
let n = length(&s); // &s = udhaar diya
println!("{}", n); // ✅ s abhi bhi tumhara hai
```

`&mut` — Likhne ke liye udhaar (mutable borrow)

```
fn add_one(v: &mut Vec<i32>) {
    v.push(1);
}

let mut v = vec![];
add_one(&mut v); // likhne ki permission ke saath udhaar
println!("{}", v); // [1]
```

Borrowing ke rules (ye yaad kar lo, life set ho jayegi)

Ya toh bahut saare log padh sakte hain (`&`), YA ek banda likh sakta hai (`&mut`). Dono ek saath **NAHI**.

Library analogy: Ek book ko 10 log ek saath padh sakte hain. Par agar koi us book mein likh raha hai, toh us time koi aur use haath nahi laga sakta.

```
let mut v = vec![1, 2, 3];
let first = &v[0]; // padhne wala borrow
v.push(4); // ❌ ERROR: likhna chahte ho jab koi padh raha hai
println!("{}", first);
```

Kyun error? `push` karne se `Vec` ki memory shift ho sakti hai, aur `first` kisi purani jagah ko point karega → crash. C++ mein ye silently bug banta. Rust pehle hi rok deta hai. 🛡️

Fix — value copy kar lo:

```
let first = v[0];    // i32 hai, copy ho gaya, borrow nahi
v.push(4);          // ✅
```

DSA ke liye function signatures — ye 3 pattern yaad rakho

```
fn read_only(v: &Vec<i32>) { }           // sirf padhna hai → &
fn modify(v: &mut Vec<i32>) { }         // change karna hai → &mut
fn take(v: Vec<i32>) { }                 // ownership chahiye (rare in DSA)

// Pro version: &Vec<i32> ki jagah &[i32] (slice) — dono chalega, slice zyada flexible hai
fn sum(v: &[i32]) -> i32 { v.iter().sum() }
```

Recursive DFS ka standard pattern:

```
fn dfs(node: usize, adj: &Vec<Vec<usize>>, visited: &mut Vec<bool>) {
    visited[node] = true;
    for &next in &adj[node] {
        if !visited[next] {
            dfs(next, adj, visited); // already reference hain, dobara & nahi lagana
        }
    }
}

// call karte waqt:
dfs(0, &adj, &mut visited);
```

* — dereference (reference ke andar ki value)

```
let mut x = 5;
let r = &mut x;
*r += 1;           // r jahan point kar raha hai, wahan +1
println!("{}", r); // 6

for x in v.iter_mut() {
    *x *= 2;        // har element double
}
```

🧠 **Mental model:** Jab compiler borrow pe chillaye, khud se poocho: "Kya main kisi cheez ko padh bhi raha hoon aur usi time badal bhi raha hoon?" 90% time answer haan hota hai. Fix: value pehle copy/clone kar lo, phir modify karo.

Part 3 — Real Rust

7. Strings — sabse zyada confuse karne wali cheez

Rust mein **2 type ke string** hain:

	String	&str
Kya hai	Owned, growable (heap pe)	Borrowed view / slice
Analogy	Tumhari apni notebook	Kisi ki notebook ka photo
Banate kaise	<code>String::from("hi")</code> , <code>"hi".to_string()</code>	<code>"hi"</code> (literal), <code>&my_string</code>
Modify	✅ (mut ho toh)	❌

```
let s1: &str = "hello";
let mut s2: String = String::from("hello");
s2.push('!');           // char add
s2.push_str(" world");  // string add
s2 += " again";

let s3: &str = &s2;      // String → &str (free, sasta)
let s4: String = s1.to_string(); // &str → String
```

Function parameter mein `&str` lo — dono type accept ho jayenge:

```
fn greet(name: &str) { println!("Hi {name}"); }
greet("Sanju");           // &str
greet(&String::from("Raj")); // &String bhi chalega
```

⚠️ **BADA GOTCHA:** `s[i]` nahi chalta!

```
let s = String::from("hello");
let c = s[0]; // ❌ ERROR: String cannot be indexed by integer
```

Kyun? Rust strings UTF-8 hain. "नमस्ते" ka ek character 3 bytes ka hota hai. Toh `s[0]` ambiguous hai.

✅ **DSA ka solution — 2 tareeke**

Tareeka 1: Bytes (fastest, jab sirf ASCII ho — 99% DSA problems):

```
let s = String::from("hello");
let b = s.as_bytes();           // &[u8]
let first = b[0];               // b'h' (u8 = 104)
if b[0] == b'h' { }             // compare byte se byte
let idx = (b[0] - b'a') as usize; // 'h' → 7
let ch = b[0] as char;          // wapas char
```

Tareeka 2: Vec (jab char methods chahiye):

```
let s = String::from("hello");
let chars: Vec<char> = s.chars().collect();
let first = chars[0];           // 'h'
if chars[0].is_alphabetic() { }
```

Common string operations

```
let s = "Hello World";

s.len()                // bytes ki length (ASCII mein = char count)
s.chars().count()      // asli characters count
s.to_lowercase()       // "hello world" (naya String)
s.to_uppercase()
s.contains("World")     // true
s.starts_with("He")    // true
s.split(' ').collect::
```

Palindrome check — sab milake

```
fn is_palindrome(s: &str) -> bool {
    let b = s.as_bytes();
    let (mut l, mut r) = (0, b.len());
    while l + 1 < r { // r ko exclusive rakha – usize underflow ka darr nahi
        if b[l] != b[r - 1] { return false; }
        l += 1;
        r -= 1;
    }
    true
}
```

8. Vec — tumhara best friend

`Vec<T>` = C++ ka `vector` , Python ka `list` . DSA mein 80% kaam isi se hoga.

Banana

```
let mut v: Vec<i32> = Vec::new();
let v = vec![1, 2, 3];
let v = vec![0; n];           // n zeros
let v = vec![-1i64; n];       // n times -1 (i64)
let grid = vec![vec![0; m]; n]; // 2D: n rows, m cols
let dp = vec![vec![vec![0; k]; m]; n]; // 3D
let v: Vec<usize> = (0..n).collect(); // [0, 1, ..., n-1]
```

Basic operations

```
let mut v = vec![3, 1, 4];

v.push(5);           // end mein add
v.pop();             // end se hatao → Option<i32> (khaali ho toh None)
v.len();             // size
v.is_empty();
v[0];               // index (out of bound → panic)
v.get(10);          // safe index → Option<&i32>
v.first(); v.last(); // Option<&i32>
v.insert(1, 9);      // index 1 pe 9 daalo (O(n))
v.remove(1);         // index 1 hatao (O(n))
v.swap(0, 2);
v.reverse();
v.clear();
v.contains(&3);       // ⚠ & lagana padta hai
v.extend(vec![7, 8]); // doosra vec jodo
v.truncate(2);        // pehle 2 rakho
v.retain(|&x| x > 2);  // sirf wo rakho jo condition pass kare
v.dedup();           // consecutive duplicates hatao (sort ke baad = unique)
v.fill(0);           // sab 0
```

Sorting 🔥

```
v.sort(); // ascending
v.sort_unstable(); // faster (DSA mein yahi use karo)
v.sort_by(|a, b| b.cmp(a)); // descending
v.sort_by_key(|&x| std::cmp::Reverse(x)); // descending (doosra tareeka)
v.sort_by_key(|x| x.abs()); // kisi key pe sort

// Pairs / intervals sort
let mut pairs = vec![(1, 'b'), (1, 'a'), (0, 'z')];
pairs.sort(); // tuple automatically lexicographic sort
pairs.sort_by_key(|p| p.1); // second element pe
pairs.sort_by(|a, b| a.0.cmp(&b.0).then(b.1.cmp(&a.1))); // first asc, phir second desc

// Vec<Vec<i32>> (LeetCode intervals)
intervals.sort_by_key(|x| x[0]);

// ⚠️ f64 seedha sort NAHI hota (NaN ki wajah se)
let mut f = vec![2.5, -1.0, 3.0];
f.sort_by(|a, b| a.total_cmp(b)); // ✅
```

Min / Max / Sum

```
let v = vec![3, 1, 4];
let mx = *v.iter().max().unwrap(); // 4 (max() Option<i32> deta hai)
let mn = *v.iter().min().unwrap(); // 1
let s: i32 = v.iter().sum(); // 8
let s64: i64 = v.iter().map(|&x| x as i64).sum(); // overflow-safe sum

let a = 5.max(3); // 5 - do numbers ka max
let b = 5.min(3); // 3
let c = (-5i32).abs();
```

Binary search (sorted Vec pe)

```
let v = vec![1, 3, 3, 5, 8];

v.binary_search(&5) // Ok(3) → mila, index 3
v.binary_search(&4) // Err(3) → nahi mila, yahan insert hota

// 🔥 lower_bound / upper_bound - DSA ke liye best
let lb = v.partition_point(|&x| x < 3); // 1 (pehla index jahan x >= 3)
let ub = v.partition_point(|&x| x <= 3); // 3 (pehla index jahan x > 3)
let count_of_3 = ub - lb; // 2
```

Slices — Vec ka ek hissa

```
let v = vec![1, 2, 3, 4, 5];
let part = &v[1..4];    // [2, 3, 4]
let tail = &v[2..];      // [3, 4, 5]
let head = &v[..2];      // [1, 2]

let (left, right) = v.split_at(2); // ([1,2], [3,4,5])
```

2D grid traverse

```
let grid = vec![vec![1, 2, 3], vec![4, 5, 6]];
let (n, m) = (grid.len(), grid[0].len());
for i in 0..n {
    for j in 0..m {
        print!("{}", grid[i][j]);
    }
    println();
}
```

9. Iterators — Rust ka superpower

Iterator = ek conveyor belt 🏢. Items ek-ek karke aate hain, tum beech mein unpe operations lagate ho, end mein `collect()` karke dabba bhar lete ho.

```
let v = vec![1, 2, 3, 4, 5];

// 3 tareeke iterate karne ke:
v.iter()      // &T deta hai (padhne ke liye)
v.iter_mut()  // &mut T (modify ke liye)
v.into_iter() // T deta hai (v consume ho jata hai, baad mein use nahi kar sakte)
```

Sabse kaam ke iterator methods

```
let v = vec![1, 2, 3, 4, 5];

// map – har element transform karo
let doubled: Vec<i32> = v.iter().map(|&x| x * 2).collect(); // [2,4,6,8,10]

// filter – sirf kuch rakho
let evens: Vec<i32> = v.iter().filter(|&x| x % 2 == 0).cloned().collect(); // [2,4]
let evens: Vec<i32> = v.iter().copied().filter(|x| x % 2 == 0).collect(); // same, cleaner ✨

// sum, count, max, min
let total: i32 = v.iter().sum();
let cnt = v.iter().filter(|&x| x > 2).count(); // 3

// any / all
v.iter().any(|&x| x > 4); // true
v.iter().all(|&x| x > 0); // true

// position – pehla index jahan condition true
v.iter().position(|&x| x == 3); // Some(2)

// enumerate – index ke saath
for (i, &x) in v.iter().enumerate() { }

// zip – do lists ko saath chalao
let a = vec![1, 2, 3];
let b = vec![4, 5, 6];
let dot: i32 = a.iter().zip(b.iter()).map(|(x, y)| x * y).sum(); // 32

// windows – sliding window of size k 🔥
let sums: Vec<i32> = v.windows(2).map(|w| w[0] + w[1]).collect(); // [3,5,7,9]

// chunks – k-k ke tukde
for c in v.chunks(2) { } // [1,2], [3,4], [5]

// rev, skip, take
v.iter().rev();
v.iter().skip(1).take(3); // 2,3,4

// fold – reduce
let product = v.iter().fold(1, |acc, &x| acc * x); // 120

// prefix sum ek line mein 😊
let prefix: Vec<i32> = v.iter().scan(0, |s, &x| { *s += x; Some(*s) }).collect(); // [1,3,6,10,15]

// max_by_key / min_by_key
let words = vec!["hi", "hello", "hey"];
let longest = words.iter().max_by_key(|w| w.len()); // Some("hello")
```

|&x| vs |x| vs ||&x|| – confusion clear karo

- `v.iter()` har item `&i32` deta hai. Toh `|&x|` likhne se `x` seedha `i32` ban jata hai (pattern se `&` hata diya).
- `filter` item ka **reference** deta hai, toh `iter() + filter = &&i32` → `|&&x|`.
- **Shortcut:** confusion ho toh `.copied()` laga do pehle — phir sab `i32` hoga, `|x|` likho, tension khatam.

```
v.iter().copied().filter(|x| *x > 2).map(|x| x * 10).collect::
```

collect ko type batana padta hai

```
let a: Vec<i32> = ...collect();           // tareeka 1: variable pe type
let a = ....collect::
```

10. Option aur Result — null ka safe version

Rust mein **null** hai hi nahi. Uski jagah **Option** hai:

```
enum Option<T> {
    Some(T),    // value hai
    None,       // value nahi hai
}
```

Analogy: Ek gift box 📦. Ya toh andar kuch hai (**Some**), ya khaali hai (**None**). Kholne se pehle check karna padega.

```
let v = vec![1, 2, 3];
let x: Option<i32> = v.get(10);    // None
let y = v.iter().max();           // Some(&3)
```

Option kholne ke tareeke

```
let opt: Option<i32> = Some(5);

// 1. unwrap – "pakka hai value hai" (None hua toh crash ✨)
let a = opt.unwrap();

// 2. unwrap_or – default value
let b = opt.unwrap_or(0);

// 3. if let – sirf Some ho tab kaam karo ☆ sabse common
if let Some(val) = opt {
    println!("mila: {val}");
}

// 4. match – dono case handle karo
match opt {
    Some(val) => println!("{val}"),
    None => println!("kuch nahi"),
}

// 5. while let – jab tak Some aata rahe (stack/queue ke liye perfect!)
let mut stack = vec![1, 2, 3];
while let Some(top) = stack.pop() {
    println!("{top}"); // 3, 2, 1
}

// 6. is_some / is_none
if opt.is_some() { }

// 7. map – andar ki value transform karo
let doubled = opt.map(|x| x * 2); // Some(10)

// 8. let-else – None ho toh turant bahar nikal jao
let Some(val) = opt else { return; };
```

🔥 **DSA mein:** `unwrap()` free mein use karo jab tumhe 100% pata ho value hai (jaise non-empty vec ka max).
Production code mein carefully.

Result — success ya error

```
enum Result<T, E> {
    Ok(T),
    Err(E),
}

let n: Result<i32, _> = "42".parse::<i32>(); // Ok(42)
let m = "abc".parse::<i32>(); // Err(...)

let x = "42".parse::<i32>().unwrap(); // DSA mein bas itna kaafi hai
```

? operator — error aaya toh function se turant return:

```
fn read_num(s: &str) -> Result<i32, std::num::ParseIntError> {
    let n = s.parse::<i32>()?; // error hua toh yahin se return Err
    Ok(n * 2)
}
```

DSA mein `Result` kam hi dikhega — bas `.unwrap()` maaro aur aage badho.

11. `match` — switch-case ka baap

```
let n = 3;
match n {
    1 => println!("one"),
    2 | 3 => println!("two or three"), // OR
    4..=9 => println!("4 se 9"), // range
    x if x < 0 => println!("negative"), // guard condition
    _ => println!("kuch aur"), // default (ZAROORI — saare cases cover karne padte hain)
}

// match bhi value deta hai
let label = match n % 3 {
    0 => "fizz",
    _ => "no fizz",
};

// tuples pe match — FizzBuzz 🤖
for i in 1..=15 {
    let s = match (i % 3, i % 5) {
        (0, 0) => "FizzBuzz".to_string(),
        (0, _) => "Fizz".to_string(),
        (_, 0) => "Buzz".to_string(),
        _ => i.to_string(),
    };
    println!("{}", s);
}

// chars pe match — bracket matching mein kaam aayega
match c {
    '(' | '[' | '{' => stack.push(c),
    ')' => { /* ... */ }
    _ => {}
}
```

12. Struct, impl, enum, trait

Struct — apna data type

```
#[derive(Debug, Clone, Copy, PartialEq)] // free features: print, clone, compare
struct Point {
    x: i32,
    y: i32,
}

impl Point {
    // constructor (convention: new)
    fn new(x: i32, y: i32) -> Self {
        Point { x, y }
    }

    // method — &self = padhne ke liye
    fn dist(&self) -> i32 {
        self.x.abs() + self.y.abs()
    }

    // method — &mut self = change karne ke liye
    fn shift(&mut self, dx: i32) {
        self.x += dx;
    }
}

let mut p = Point::new(3, -4);
println!("{}", p.dist()); // 7
p.shift(1);
println!("{:?}", p); // Point { x: 4, y: -4 }
```

`#[derive(...)]` — free mein superpowers

derive	Kya milta hai
Debug	<code>{:?}</code> se print
Clone	<code>.clone()</code>
Copy	Auto copy (sirf jab saare fields Copy hon)
PartialEq, Eq	<code>==</code> compare
PartialOrd, Ord	<code><</code> , <code>></code> , sort, BinaryHeap mein daal sakte ho
Hash	HashMap/HashSet ka key ban sakta hai
Default	<code>Point::default()</code> → sab zero

🔥 DSA mein apna struct HashMap key ya heap mein daalna ho → `#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord)]` laga do, sab kuch chalega.

Enum — "ye ya wo" type

```
enum Dir { Up, Down, Left, Right }

fn delta(d: Dir) -> (i32, i32) {
    match d {
        Dir::Up => (-1, 0),
        Dir::Down => (1, 0),
        Dir::Left => (0, -1),
        Dir::Right => (0, 1),
    }
}

// Enum andar data bhi rakh sakta hai
enum Shape {
    Circle(f64),
    Rect { w: f64, h: f64 },
}
```

Trait — interface jaisa

```
trait Area {
    fn area(&self) -> f64;
}

impl Area for Shape {
    fn area(&self) -> f64 {
        match self {
            Shape::Circle(r) => 3.14 * r * r,
            Shape::Rect { w, h } => w * h,
        }
    }
}
```

DSA mein trait khud likhna rare hai — bas `derive` use karoge.

13. Closures — chhote anonymous functions

```
let add = |a: i32, b: i32| a + b;
println!("{}", add(2, 3)); // 5

let sq = |x| x * x; // type khud guess

// bahar ke variable use kar sakta hai
let k = 10;
let add_k = |x: i32| x + k;

// sort, filter, map mein yahi to use ho raha tha!
v.sort_by_key(|x| x.abs());
```

⚠ **Closure recursive nahi ho sakta** (aasaani se). DFS/recursion ke liye normal `fn` likho, aur jo cheezein chahiye wo parameters mein pass karo. Ye Rust DSA ka standard tareeka hai.

Nested fn chalta hai (bas wo bahar ke variables nahi dekh sakta):

```
fn solve(grid: Vec<Vec<i32>>)> -> i32 {  
    fn dfs(g: &Vec<Vec<i32>>, i: usize) -> i32 { 0 } // ✅ andar define kar sakte ho  
    dfs(&grid, 0)  
}
```

14. Collections: HashMap, HashSet, BTreeMap, VecDeque, BinaryHeap

Sabse pehle top pe import:

```
use std::collections::{HashMap, HashSet, BTreeMap, BTreeSet, VecDeque, BinaryHeap};  
use std::cmp::Reverse;
```

HashMap — key → value

```
let mut mp: HashMap<String, i32> = HashMap::new();

mp.insert("apple".to_string(), 3);
mp.get("apple");           // Option<&i32> → Some(&3)
mp.contains_key("apple");  // true
mp.remove("apple");
mp.len();

// 🔥 FREQUENCY COUNT — ye pattern 100 baar use hoga
let mut freq: HashMap<char, i32> = HashMap::new();
for c in "banana".chars() {
    *freq.entry(c).or_insert(0) += 1;
}
// freq = {'b': 1, 'a': 3, 'n': 2}

// Default value ke saath get
let count = *freq.get(&'z').unwrap_or(&0);    // 0

// Value modify karo
if let Some(v) = freq.get_mut(&'a') {
    *v += 10;
}

// Group anagrams style — key → list
let mut groups: HashMap<String, Vec<String>> = HashMap::new();
groups.entry("key".to_string()).or_insert_with(Vec::new).push("val".to_string());
// ya: .or_default().push(...)

// Iterate
for (key, val) in &freq {
    println!("{key}: {val}");
}
for k in freq.keys() { }
for v in freq.values() { }
```

⚠️ HashMap ka order **random** hota hai. Sorted chahiye → BTreeMap .

HashSet — unique items

```
let mut seen: HashSet<i32> = HashSet::new();
seen.insert(5);           // true (naya tha)
seen.insert(5);           // false (pehle se tha)
seen.contains(&5);        // true
seen.remove(&5);

let set: HashSet<i32> = vec![1, 2, 2, 3].into_iter().collect();    // {1,2,3}

// set operations
let a: HashSet<i32> = [1, 2, 3].into_iter().collect();
let b: HashSet<i32> = [2, 3, 4].into_iter().collect();
let common: Vec<_> = a.intersection(&b).collect();    // [2,3]
let all: Vec<_> = a.union(&b).collect();
```

BTreeMap / BTreeSet — sorted (C++ ka map/set)

```
let mut bt = BTreeMap::new();
bt.insert(5, "five");
bt.insert(2, "two");
bt.insert(8, "eight");

bt.first_key_value(); // Some((2, "two")) - smallest
bt.last_key_value(); // Some((8, "eight")) - largest
bt.range(3..).next(); // Some((5, "five")) - pehla key >= 3 (lower_bound!)
bt.range(..5).next_back(); // Some((2, "two")) - sabse bada key < 5

let mut bs = BTreeSet::new();
bs.insert(3); bs.insert(1);
bs.first(); // Some(&1)
bs.pop_first(); // nikaal ke do
```

VecDeque — dono taraf se queue (BFS ke liye)

```
let mut q = VecDeque::new();
q.push_back(1); // peeche daalo
q.push_front(0); // aage daalo
q.pop_front(); // Option - aage se nikalo (BFS mein yahi)
q.pop_back();
q.front(); q.back(); // peek
q.is_empty();
```

BinaryHeap — priority queue 🔥

Default MAX-heap hai (sabse bada pehle nikalta hai).

```
let mut pq = BinaryHeap::new();
pq.push(3);
pq.push(10);
pq.push(1);
pq.peek(); // Some(&10)
pq.pop(); // Some(10)

// MIN-heap chahiye? Reverse() mein lapet do
let mut min_pq = BinaryHeap::new();
min_pq.push(Reverse(3));
min_pq.push(Reverse(1));
if let Some(Reverse(smallest)) = min_pq.pop() {
    println!("{smallest}"); // 1
}

// Tuples bhi daal sakte ho - pehle element pe compare hoga
let mut pq = BinaryHeap::new();
pq.push(Reverse((dist, node))); // Dijkstra style
```

Kaunsa kab? Quick table

Chahiye	Use karo	C++ equivalent
List / array	<code>Vec</code>	<code>vector</code>
Stack	<code>Vec</code> (push/pop)	<code>stack</code>
Queue	<code>VecDeque</code>	<code>queue</code> / <code>deque</code>
Key-value (fast)	<code>HashMap</code>	<code>unordered_map</code>
Unique set (fast)	<code>HashSet</code>	<code>unordered_set</code>
Sorted key-value	<code>BTreeMap</code>	<code>map</code>
Sorted set	<code>BTreeSet</code>	<code>set</code>
Max heap	<code>BinaryHeap</code>	<code>priority_queue</code>
Min heap	<code>BinaryHeap<Reverse<T>></code>	<code>priority_queue<T, vector<T>, greater<T>></code>

Part 4 — DSA Mode ON 🔥

15. LeetCode aur Codeforces template

LeetCode format

LeetCode tumhe ye dega — bas function ke andar likhna hai:

```
impl Solution {
    pub fn two_sum(nums: Vec<i32>, target: i32) -> Vec<i32> {
        use std::collections::HashMap;
        let mut seen: HashMap<i32, usize> = HashMap::new();
        for (i, &x) in nums.iter().enumerate() {
            if let Some(&j) = seen.get(&(target - x)) {
                return vec![j as i32, i as i32];
            }
            seen.insert(x, i);
        }
        vec![]
    }
}
```

Tip: Parameter ko modify karna hai (jaise `nums` sort karna)? Signature mein `mut` laga do: `pub fn f(mut nums: Vec<i32>)` . LeetCode allow karta hai.

Codeforces / CodeChef — fast I/O template

Ye template copy karke rakh lo. Input poora ek saath padhta hai, bahut fast hai:

```

use std::io::{self, Read, Write, BufWriter};

struct Scanner<'a> {
    it: std::str::SplitAsciiWhitespace<'a>,
}

impl<'a> Scanner<'a> {
    fn new(s: &'a str) -> Self {
        Scanner { it: s.split_ascii_whitespace() }
    }
    fn next<T: std::str::FromStr>(&mut self) -> T
    where
        T::Err: std::fmt::Debug,
    {
        self.it.next().unwrap().parse().unwrap()
    }
}

fn main() {
    let mut input = String::new();
    io::stdin().read_to_string(&mut input).unwrap();
    let mut sc = Scanner::new(&input);
    let mut out = BufWriter::new(io::stdout().lock());

    let t: usize = sc.next();
    for _ in 0..t {
        let n: usize = sc.next();
        let a: Vec<i64> = (0..n).map(|_| sc.next()).collect();
        let s: String = sc.next(); // ek word

        let ans: i64 = a.iter().sum();
        writeln!(out, "{}", ans).unwrap();
    }
}

```

Kya ho raha hai:

- `sc.next()` — agla token padho. Type left side se decide hota hai: `let n: usize = sc.next();`
- `BufWriter` + `writeln!` — output fast. `println!` ko loop mein lakhon baar call karoge toh TLE aa sakta hai.
- `Vec` ko space-separated print karna:

```

let line: Vec<String> = a.iter().map(|x| x.to_string()).collect();
writeln!(out, "{}", line.join(" ")).unwrap();

```

16. DSA patterns cookbook

Har pattern ka ready-made Rust code. Logic tumhe pata hai, syntax yahan se utha lo. 📄

16.1 Two Pointers

```
// Sorted array mein pair with sum = target
fn two_sum_sorted(v: &[i32], target: i32) -> Option<(usize, usize)> {
    if v.is_empty() { return None; }
    let (mut l, mut r) = (0, v.len() - 1);
    while l < r {
        let s = v[l] + v[r];
        if s == target { return Some((l, r)); }
        if s < target { l += 1; } else { r -= 1; }
    }
    None
}
```

16.2 Sliding Window

```
// Longest substring without repeating characters
fn length_of_longest_substring(s: String) -> i32 {
    let s = s.as_bytes();
    let mut cnt = [0; 128];
    let (mut l, mut best) = (0, 0);
    for r in 0..s.len() {
        cnt[s[r] as usize] += 1;
        while cnt[s[r] as usize] > 1 {
            cnt[s[l] as usize] -= 1;
            l += 1;
        }
        best = best.max(r - l + 1);
    }
    best as i32
}

// Fixed window size k - max sum
fn max_sum_k(v: &[i64], k: usize) -> i64 {
    let mut cur: i64 = v[..k].iter().sum();
    let mut best = cur;
    for i in k..v.len() {
        cur += v[i] - v[i - k];
        best = best.max(cur);
    }
    best
}
```

16.3 Prefix Sum

```
let v = vec![3, 1, 4, 1, 5];
let n = v.len();
let mut pre = vec![0; n + 1]; // pre[i] = pehle i elements ka sum
for i in 0..n {
    pre[i + 1] = pre[i] + v[i] as i64;
}
// sum of v[l..=r] = pre[r + 1] - pre[l]
let range_sum = pre[4] - pre[1]; // v[1..=3] = 1+4+1 = 6
```

16.4 Binary Search (manual — answer pe binary search ke liye)

```
// lower_bound: pehla index jahan v[i] >= target
fn lower_bound(v: &[i32], target: i32) -> usize {
    let (mut lo, mut hi) = (0, v.len());    // [lo, hi) — half-open, usize safe
    while lo < hi {
        let mid = lo + (hi - lo) / 2;
        if v[mid] < target { lo = mid + 1; } else { hi = mid; }
    }
    lo
}

// Binary search on answer (e.g. Koko eating bananas)
fn min_speed(piles: Vec<i32>, h: i32) -> i32 {
    let can = |k: i64| -> bool {
        piles.iter().map(|&p| (p as i64 + k - 1) / k).sum::<i64>() <= h as i64
    };
    let (mut lo, mut hi) = (1i64, *piles.iter().max().unwrap() as i64);
    while lo < hi {
        let mid = lo + (hi - lo) / 2;
        if can(mid) { hi = mid; } else { lo = mid + 1; }
    }
    lo as i32
}
```

💡 [lo, hi) half-open style use karo — hi = mid - 1 jaisa kuch nahi likhna padta, toh usize underflow (0 - 1) ka darr khatam.

16.5 Stack — valid parentheses

```
fn is_valid(s: String) -> bool {
    let mut st: Vec<char> = Vec::new();
    for c in s.chars() {
        match c {
            '(' => st.push('('),
            '[' => st.push('['),
            '{' => st.push('{'),
            _ => {
                if st.pop() != Some(c) { return false; }
            }
        }
    }
    st.is_empty()
}
```

16.6 Monotonic Stack — next greater element

```
fn next_greater(v: &[i32]) -> Vec<i32> {
    let n = v.len();
    let mut ans = vec![-1; n];
    let mut st: Vec<usize> = Vec::new(); // indices store karo
    for i in 0..n {
        while let Some(&top) = st.last() {
            if v[top] < v[i] {
                ans[top] = v[i];
                st.pop();
            } else {
                break;
            }
        }
        st.push(i);
    }
    ans
}
```

16.7 BFS on Grid — shortest path

```
use std::collections::VecDeque;

fn shortest_path(grid: &Vec<Vec<i32>>) -> i32 {
    let (n, m) = (grid.len(), grid[0].len());
    let mut dist = vec![vec![-1; m]; n];
    let mut q = VecDeque::new();
    dist[0][0] = 0;
    q.push_back((0usize, 0usize));

    let dirs = [(0i32, 1i32), (1, 0), (0, -1), (-1, 0)];
    while let Some((r, c)) = q.pop_front() {
        for (dr, dc) in dirs {
            let nr = r as i32 + dr;
            let nc = c as i32 + dc;
            // bounds check i32 mein karo, phir usize mein convert
            if nr < 0 || nc < 0 || nr >= n as i32 || nc >= m as i32 { continue; }
            let (nr, nc) = (nr as usize, nc as usize);
            if grid[nr][nc] == 1 || dist[nr][nc] != -1 { continue; } // 1 = wall
            dist[nr][nc] = dist[r][c] + 1;
            q.push_back((nr, nc));
        }
    }
    dist[n - 1][m - 1]
}
```

 **Grid ka sabse bada dard:** `usize` negative nahi ho sakta, toh `r - 1` jab `r = 0` ho → panic. **Solution:** neighbour calculate karte waqt `i32` mein convert karo, check karo, phir wapas `usize`. Upar wala pattern yaad kar lo.

16.8 DFS on Grid — number of islands

```
fn num_islands(mut grid: Vec<Vec<char>>) -> i32 {
    fn dfs(g: &mut Vec<Vec<char>>, i: i32, j: i32) {
        if i < 0 || j < 0 || i >= g.len() as i32 || j >= g[0].len() as i32 { return; }
        let (r, c) = (i as usize, j as usize);
        if g[r][c] != '1' { return; }
        g[r][c] = '0'; // visited mark
        dfs(g, i + 1, j);
        dfs(g, i - 1, j);
        dfs(g, i, j + 1);
        dfs(g, i, j - 1);
    }

    let mut count = 0;
    for i in 0..grid.len() {
        for j in 0..grid[0].len() {
            if grid[i][j] == '1' {
                count += 1;
                dfs(&mut grid, i as i32, j as i32);
            }
        }
    }
    count
}
```

16.9 Graph — adjacency list, DFS, BFS

```
// edges: Vec<Vec<i32>> like [[0,1],[1,2]] (LeetCode style)
fn build_graph(n: usize, edges: &Vec<Vec<i32>>) -> Vec<Vec<usize>> {
    let mut adj = vec![vec![]; n];
    for e in edges {
        let (u, v) = (e[0] as usize, e[1] as usize);
        adj[u].push(v);
        adj[v].push(u); // undirected
    }
    adj
}

fn dfs(u: usize, adj: &Vec<Vec<usize>>, vis: &mut Vec<bool>) {
    vis[u] = true;
    for &v in &adj[u] {
        if !vis[v] { dfs(v, adj, vis); }
    }
}

// connected components count
fn count_components(n: usize, edges: Vec<Vec<i32>>) -> i32 {
    let adj = build_graph(n, &edges);
    let mut vis = vec![false; n];
    let mut c = 0;
    for i in 0..n {
        if !vis[i] { dfs(i, &adj, &mut vis); c += 1; }
    }
    c
}
```

16.10 Topological Sort (Kahn's BFS)

```
use std::collections::VecDeque;

fn topo_sort(n: usize, edges: &Vec<Vec<i32>>) -> Option<Vec<usize>> {
    let mut adj = vec![vec![]; n];
    let mut indeg = vec![0; n];
    for e in edges {
        let (u, v) = (e[0] as usize, e[1] as usize);
        adj[u].push(v);
        indeg[v] += 1;
    }
    let mut q: VecDeque<usize> = (0..n).filter(|&i| indeg[i] == 0).collect();
    let mut order = vec![];
    while let Some(u) = q.pop_front() {
        order.push(u);
        for &v in &adj[u] {
            indeg[v] -= 1;
            if indeg[v] == 0 { q.push_back(v); }
        }
    }
    if order.len() == n { Some(order) } else { None } // None = cycle hai
}
```

16.11 Dijkstra

```
use std::collections::BinaryHeap;
use std::cmp::Reverse;

// adj[u] = vec![(v, weight), ...]
fn dijkstra(adj: &Vec<Vec<(usize, i64)>>, src: usize) -> Vec<i64> {
    let mut dist = vec![i64::MAX; adj.len()];
    let mut pq = BinaryHeap::new();
    dist[src] = 0;
    pq.push(Reverse((0i64, src)));

    while let Some(Reverse((d, u))) = pq.pop() {
        if d > dist[u] { continue; } // purana entry, skip
        for &(v, w) in &adj[u] {
            let nd = d + w;
            if nd < dist[v] {
                dist[v] = nd;
                pq.push(Reverse((nd, v)));
            }
        }
    }
    dist
}
```

16.12 Union-Find (DSU)

```
struct Dsu {
    parent: Vec<usize>,
    size: Vec<usize>,
}

impl Dsu {
    fn new(n: usize) -> Self {
        Dsu { parent: (0..n).collect(), size: vec![1; n] }
    }
    fn find(&mut self, x: usize) -> usize {
        if self.parent[x] != x {
            self.parent[x] = self.find(self.parent[x]); // path compression
        }
        self.parent[x]
    }
    fn union(&mut self, a: usize, b: usize) -> bool {
        let (mut a, mut b) = (self.find(a), self.find(b));
        if a == b { return false; }
        if self.size[a] < self.size[b] { std::mem::swap(&mut a, &mut b); }
        self.parent[b] = a;
        self.size[a] += self.size[b];
        true
    }
}

// use:
let mut dsu = Dsu::new(5);
dsu.union(0, 1);
let same = dsu.find(0) == dsu.find(1); // true
```

16.13 Dynamic Programming

```
// 1D - climbing stairs
fn climb_stairs(n: i32) -> i32 {
    let n = n as usize;
    let mut dp = vec![0; n + 2];
    dp[0] = 1;
    dp[1] = 1;
    for i in 2..=n {
        dp[i] = dp[i - 1] + dp[i - 2];
    }
    dp[n]
}

// 2D - Longest Common Subsequence
fn lcs(a: String, b: String) -> i32 {
    let (a, b) = (a.as_bytes(), b.as_bytes());
    let (n, m) = (a.len(), b.len());
    let mut dp = vec![vec![0; m + 1]; n + 1];
    for i in 1..=n {
        for j in 1..=m {
            dp[i][j] = if a[i - 1] == b[j - 1] {
                dp[i - 1][j - 1] + 1
            } else {
                dp[i - 1][j].max(dp[i][j - 1])
            };
        }
    }
    dp[n][m]
}

// 0/1 Knapsack (1D optimized)
fn knapsack(wt: &[usize], val: &[i64], cap: usize) -> i64 {
    let mut dp = vec![0i64; cap + 1];
    for i in 0..wt.len() {
        for w in (wt[i]..=cap).rev() { // ulta loop!
            dp[w] = dp[w].max(dp[w - wt[i]] + val[i]);
        }
    }
    dp[cap]
}
```

16.14 Memoization (top-down DP)

```
fn fib(n: usize, memo: &mut Vec<Option<i64>>) -> i64 {
    if n <= 1 { return n as i64; }
    if let Some(v) = memo[n] { return v; }
    let ans = fib(n - 1, memo) + fib(n - 2, memo);
    memo[n] = Some(ans);
    ans
}

let mut memo = vec![None; 91];
println!("{}", fib(90, &mut memo));

// HashMap memo (jab state complex ho)
use std::collections::HashMap;
fn solve(i: usize, j: usize, memo: &mut HashMap<(usize, usize), i64>) -> i64 {
    if let Some(&v) = memo.get(&(i, j)) { return v; }
    let ans = 0; // ... calculate
    memo.insert((i, j), ans);
    ans
}
```

💡 Memo mein "not computed" ke liye `-1` bhi use kar sakte ho (`vec![-1; n]`), par `Option` zyada clean hai.

16.15 Backtracking — subsets & permutations

```
// Subsets
fn subsets(nums: Vec<i32>) -> Vec<Vec<i32>> {
    fn bt(start: usize, nums: &[i32], path: &mut Vec<i32>, res: &mut Vec<Vec<i32>>) {
        res.push(path.clone());
        for i in start..nums.len() {
            path.push(nums[i]);
            bt(i + 1, nums, path, res);
            path.pop(); // undo
        }
    }
    let mut res = vec![];
    bt(0, &nums, &mut vec![], &mut res);
    res
}

// Permutations
fn permute(nums: Vec<i32>) -> Vec<Vec<i32>> {
    fn bt(nums: &[i32], used: &mut Vec<bool>, path: &mut Vec<i32>, res: &mut Vec<Vec<i32>>) {
        if path.len() == nums.len() {
            res.push(path.clone());
            return;
        }
        for i in 0..nums.len() {
            if used[i] { continue; }
            used[i] = true;
            path.push(nums[i]);
            bt(nums, used, path, res);
            path.pop();
            used[i] = false;
        }
    }
    let mut res = vec![];
    bt(&nums, &mut vec![false; nums.len()], &mut vec![], &mut res);
    res
}
```

16.16 Intervals — merge

```
fn merge(mut intervals: Vec<Vec<i32>>) -> Vec<Vec<i32>> {
    intervals.sort_by_key(|x| x[0]);
    let mut res: Vec<Vec<i32>> = vec![];
    for it in intervals {
        if let Some(last) = res.last_mut() {
            if it[0] <= last[1] {
                last[1] = last[1].max(it[1]);
                continue;
            }
        }
        res.push(it);
    }
    res
}
```

16.17 Heap — Top K / Kth largest

```
use std::collections::BinaryHeap;
use std::cmp::Reverse;

fn find_kth_largest(nums: Vec<i32>, k: i32) -> i32 {
    let mut pq = BinaryHeap::new();           // min-heap of size k
    for x in nums {
        pq.push(Reverse(x));
        if pq.len() > k as usize { pq.pop(); }
    }
    pq.peek().unwrap().0                     // .0 = Reverse ke andar ki value
}
```

16.18 Trie (Rust-friendly "arena" style)

Box aur pointers ke saath Trie Rust mein borrow-checker se ladai karwata hai. DSA ke liye **Vec-based (arena) trie** best hai — fast bhi, simple bhi:

```
struct Trie {
    next: Vec<[usize; 26]>, // next[node][c] = child node index (0 = nahi hai)
    end: Vec<bool>,
}

impl Trie {
    fn new() -> Self {
        Trie { next: vec![0; 26], end: vec![false] } // node 0 = root
    }

    fn insert(&mut self, word: &str) {
        let mut cur = 0;
        for b in word.bytes() {
            let c = (b - b'a') as usize;
            if self.next[cur][c] == 0 {
                self.next.push([0; 26]);
                self.end.push(false);
                self.next[cur][c] = self.next.len() - 1;
            }
            cur = self.next[cur][c];
        }
        self.end[cur] = true;
    }

    fn search(&self, word: &str) -> bool {
        let mut cur = 0;
        for b in word.bytes() {
            let c = (b - b'a') as usize;
            if self.next[cur][c] == 0 { return false; }
            cur = self.next[cur][c];
        }
        self.end[cur]
    }
}
```

 **Pro tip:** Rust mein jab bhi pointer-heavy structure (tree, graph, trie, linked list) banana ho, **indices (`usize`) ko pointer ki tarah use karo** aur data `Vec` mein rakho. Borrow checker khush, tum khush.

16.19 Bit manipulation

```
let x: u32 = 0b1011;
x.count_ones()      // 3 (set bits)
x.trailing_zeros()   // 0
x.leading_zeros()
x & (x - 1)          // lowest set bit hatao
x & x.wrapping_neg() // sirf lowest set bit
1 << k               // 2^k
(x >> k) & 1         // k-th bit
x ^ y                // XOR

// Saare subsets via bitmask
let n = 3;
for mask in 0..(1 << n) {
    let subset: Vec<usize> = (0..n).filter(|&i| mask & (1 << i) != 0).collect();
}
```

16.20 Math helpers

```
const MOD: i64 = 1_000_000_007;

fn gcd(a: i64, b: i64) -> i64 {
    if b == 0 { a } else { gcd(b, a % b) }
}

fn lcm(a: i64, b: i64) -> i64 { a / gcd(a, b) * b }

fn mod_pow(mut base: i64, mut exp: i64, m: i64) -> i64 {
    let mut res = 1;
    base %= m;
    while exp > 0 {
        if exp & 1 == 1 { res = res * base % m; }
        base = base * base % m;
        exp >>= 1;
    }
    res
}

// Sieve of Eratosthenes
fn sieve(n: usize) -> Vec<bool> {
    let mut is_prime = vec![true; n + 1];
    is_prime[0] = false;
    if n >= 1 { is_prime[1] = false; }
    let mut i = 2;
    while i * i <= n {
        if is_prime[i] {
            let mut j = i * i;
            while j <= n { is_prime[j] = false; j += i; }
        }
        i += 1;
    }
    is_prime
}

// Negative mod ko positive banana
let r = ((a % MOD) + MOD) % MOD;
// ya
let r = a.rem_euclid(MOD);
```

17. Linked List aur Tree (LeetCode wala dard)

Ye LeetCode ka sabse awkward part hai Rust mein. Dar mat, pattern ratt lo.

Linked List

LeetCode ye definition deta hai:

```
pub struct ListNode {
    pub val: i32,
    pub next: Option<Box<ListNode>>,
}
```

- `Box` = heap pe rakha hua data (pointer jaisa)
- `Option` = ho bhi sakta hai, nahi bhi (null ki jagah)

Reverse linked list — ye pattern sab sikha deta hai:

```
pub fn reverse_list(head: Option<Box<ListNode>>) -> Option<Box<ListNode>> {
    let mut prev = None;
    let mut curr = head;
    while let Some(mut node) = curr {
        curr = node.next.take(); // take() = value nikalo, wahan None chhod do
        node.next = prev;
        prev = Some(node);
    }
    prev
}
```

🔑 **.take() is the hero.** `Option` se value nikal leta hai aur peeche `None` chhod deta hai. Ownership ka jhagda khatam.

Traverse karna (sirf padhna):

```
let mut cur = &head;
while let Some(node) = cur {
    println!("{}", node.val);
    cur = &node.next;
}
```

Easy hack: Linked list problem bahut tough lag rahi ho → values `Vec` mein daalo, `Vec` pe solve karo, phir nayi list banao:

```
// Vec se list banana (peeche se)
let mut head: Option<Box<ListNode>> = None;
for &v in vals.iter().rev() {
    head = Some(Box::new(ListNode { val: v, next: head }));
}
```

Binary Tree

LeetCode definition:

```
use std::rc::Rc;
use std::cell::RefCell;

pub struct TreeNode {
    pub val: i32,
    pub left: Option<Rc<RefCell<TreeNode>>>,
    pub right: Option<Rc<RefCell<TreeNode>>>,
}
```

Daro mat! Bas itna samjho:

- `Rc` = shared pointer (multiple owners ho sakte hain)
- `RefCell` = andar ki value ko `.borrow()` (padhna) ya `.borrow_mut()` (likhna) se access karo

Pattern: `&Option<Rc<RefCell<TreeNode>>>` reference lo, `if let Some(node)` se kholo, `.borrow()` se padho.

```

// Max depth
fn max_depth(root: Option<Rc<RefCell<TreeNode>>>) -> i32 {
    fn depth(node: &Option<Rc<RefCell<TreeNode>>>) -> i32 {
        match node {
            None => 0,
            Some(n) => {
                let n = n.borrow();
                1 + depth(&n.left).max(depth(&n.right))
            }
        }
    }
    depth(&root)
}

// Inorder traversal
fn inorder_traversal(root: Option<Rc<RefCell<TreeNode>>>) -> Vec<i32> {
    fn go(node: &Option<Rc<RefCell<TreeNode>>>, out: &mut Vec<i32>) {
        if let Some(n) = node {
            let n = n.borrow();
            go(&n.left, out);
            out.push(n.val);
            go(&n.right, out);
        }
    }
    let mut out = vec![];
    go(&root, &mut out);
    out
}

// Level order (BFS)
fn level_order(root: Option<Rc<RefCell<TreeNode>>>) -> Vec<Vec<i32>> {
    use std::collections::VecDeque;
    let mut res = vec![];
    let mut q = VecDeque::new();
    if let Some(r) = root { q.push_back(r); }
    while !q.is_empty() {
        let mut level = vec![];
        for _ in 0..q.len() {
            let node = q.pop_front().unwrap();
            let n = node.borrow();
            level.push(n.val);
            if let Some(l) = n.left.clone() { q.push_back(l); }
            if let Some(r) = n.right.clone() { q.push_back(r); }
        }
        res.push(level);
    }
    res
}

// Invert tree (modify karna - borrow_mut)
fn invert_tree(root: Option<Rc<RefCell<TreeNode>>>) -> Option<Rc<RefCell<TreeNode>>> {
    if let Some(node) = root.clone() {
        let mut n = node.borrow_mut();
        let l = n.left.take();
        let r = n.right.take();
        n.left = invert_tree(r);
        n.right = invert_tree(l);
    }
}

```

```
    root
}
```

💡 Rc ka `.clone()` **sasta** hai — sirf pointer copy hota hai, poora tree nahi. Toh `n.left.clone()` freely karo.

18. Common errors aur unka ilaaj

Error message (short)	Matlab	Ilaaj 🍌
use of moved value / borrow of moved value	Value kisi aur ko de chuke ho	& se borrow karo, ya <code>.clone()</code>
cannot borrow as mutable because it is also borrowed as immutable	Padh bhi rahe ho, likh bhi rahe ho	Value pehle copy karo: <code>let x = v[i];</code> phir modify
cannot borrow as mutable (variable pe)	mut bhool gaye	<code>let mut x</code>
attempt to subtract with overflow (panic)	usize 0 se neeche gaya	i32 mein calculate karo, ya <code>if i > 0</code> check, ya <code>[lo, hi)</code> style
attempt to add with overflow	i32 overflow	i64 use karo
mismatched types: expected usize, found i32	Type mix	<code>as usize</code> / <code>as i32</code>
expected i32, found ()	Last line pe <code>;</code> laga diya	Return wali line se <code>;</code> hatao
the type String cannot be indexed by {integer}	<code>s[i]</code> kiya	<code>s.as_bytes()[i]</code> ya <code>Vec<char></code>
the trait Ord is not implemented for f64	float sort	<code>sort_by(a, b a.total_cmp(b))</code>
index out of bounds (panic)	Galat index	<code>v.len()</code> check karo, ya <code>v.get(i)</code>
called Option::unwrap() on a None value	None ko unwrap kiya	<code>if let Some(x)</code> ya <code>unwrap_or</code>
cannot borrow *self as mutable more than once	Struct method mein double borrow	Values local variable mein nikalo pehle
type annotations needed	<code>collect()</code> / <code>parse()</code> ko type nahi pata	<code>: Vec<i32></code> ya <code>::<i32></code>
expected &i32, found i32 (ya ulta)	Reference ka chakkar	<code>&x</code> lagao ya <code>*x</code> se deref karo

Usize underflow — sabse common DSA bug, 3 fix

```
// ❌ i = 0 pe crash
for i in 0..n {
    if v[i - 1] < v[i] { }
}

// ✅ Fix 1: range hi 1 se shuru karo
for i in 1..n { if v[i - 1] < v[i] { } }

// ✅ Fix 2: checked_sub
if let Some(prev) = i.checked_sub(1) { /* v[prev] */ }

// ✅ Fix 3: n - 1 jab n = 0 ho sakta hai
for i in 0..n.saturating_sub(1) { } // n=0 pe 0 deta hai, crash nahi
```

"Same vec mein se padh ke likhna" error

```
v.push(v[0]); // ✅ chal jata hai — v[0] i32 hai, pehle copy hota hai, phir push

// ❌ ERROR — reference rakh liya, phir push kiya
let first = &v[0];
v.push(*first); // borrow ab bhi zinda hai

// ✅
let first = v[0];
v.push(first);

// 2D grid mein:
grid[i][j] = grid[i - 1][j] + grid[i][j - 1]; // ✅ ye chal jata hai (numbers Copy hain)
```

19. C++/Python → Rust cheat sheet

Kaam	C++	Python	Rust
Variable	<code>int x = 5;</code>	<code>x = 5</code>	<code>let mut x = 5;</code>
Constant	<code>const int N = 5;</code>	<code>N = 5</code>	<code>const N: usize = 5;</code>
Print	<code>cout << x;</code>	<code>print(x)</code>	<code>println!("{}", x);</code>
Array of n zeros	<code>vector<int> v(n);</code>	<code>[0]*n</code>	<code>vec![0; n]</code>
2D	<code>vector<vector<int>> g(n, vector<int>(m));</code>	<code>[[0]*m for _ in range(n)]</code>	<code>vec![vec![0; m]; n]</code>
Push	<code>v.push_back(x)</code>	<code>v.append(x)</code>	<code>v.push(x)</code>
Size	<code>v.size()</code>	<code>len(v)</code>	<code>v.len()</code>
Loop	<code>for(int i=0;i<n;i++)</code>	<code>for i in range(n)</code>	<code>for i in 0..n</code>
Reverse loop	<code>for(int i=n-1;i>=0;i--)</code>	<code>for i in range(n-1,-1,-1)</code>	<code>for i in (0..n).rev()</code>

Kaam	C++	Python	Rust
Foreach	<code>for(auto x : v)</code>	<code>for x in v</code>	<code>for &x in &v / for x in &v</code>
Sort	<code>sort(v.begin(), v.end())</code>	<code>v.sort()</code>	<code>v.sort()</code>
Sort desc	<code>sort(..., greater<int>())</code>	<code>v.sort(reverse=True)</code>	<code>v.sort_by(a, b b.cmp(a))</code>
Reverse	<code>reverse(v.begin(), v.end())</code>	<code>v.reverse()</code>	<code>v.reverse()</code>
Max of 2	<code>max(a, b)</code>	<code>max(a, b)</code>	<code>a.max(b)</code>
Max of vec	<code>*max_element(...)</code>	<code>max(v)</code>	<code>*v.iter().max().unwrap()</code>
Sum	<code>accumulate(...)</code>	<code>sum(v)</code>	<code>v.iter().sum::<i32>()</code>
Swap	<code>swap(a, b)</code>	<code>a, b = b, a</code>	<code>std::mem::swap(&mut a, &mut b) / v.swap(i, j)</code>
INF	<code>INT_MAX / LLONG_MAX</code>	<code>float('inf')</code>	<code>i32::MAX / i64::MAX</code>
Map	<code>unordered_map<K,V></code>	<code>dict</code>	<code>HashMap<K, V></code>
Freq++	<code>mp[k]++</code>	<code>d[k] = d.get(k,0)+1</code>	<code>*mp.entry(k).or_insert(0) += 1</code>
Set	<code>unordered_set</code>	<code>set()</code>	<code>HashSet</code>
Queue	<code>queue</code>	<code>deque</code>	<code>VecDeque</code>
Stack	<code>stack</code>	<code>list</code>	<code>Vec</code>
Max heap	<code>priority_queue<int></code>	<code>heapq (negate)</code>	<code>BinaryHeap</code>
Min heap	<code>priority_queue<int,vector<int>,greater<int>></code>	<code>heapq</code>	<code>BinaryHeap<Reverse<i32>></code>
Pair	<code>pair<int,int></code>	<code>(a, b)</code>	<code>(i32, i32)</code>
Lower bound	<code>lower_bound(...)</code>	<code>bisect_left</code>	<code>v.partition_point(&x x < t)</code>
Upper bound	<code>upper_bound(...)</code>	<code>bisect_right</code>	<code>v.partition_point(&x x <= t)</code>
String char	<code>s[i]</code>	<code>s[i]</code>	<code>s.as_bytes()[i] / chars[i]</code>
Substring	<code>s.substr(i, len)</code>	<code>s[i:i+len]</code>	<code>&s[i..i+len]</code>
Str → int	<code>stoi(s)</code>	<code>int(s)</code>	<code>s.parse::<i32>().unwrap()</code>
Int → str	<code>to_string(x)</code>	<code>str(x)</code>	<code>x.to_string()</code>
Null	<code>nullptr</code>	<code>None</code>	<code>None (Option)</code>
Abs	<code>abs(x)</code>	<code>abs(x)</code>	<code>x.abs()</code>
Ternary	<code>c ? a : b</code>	<code>a if c else b</code>	<code>if c { a } else { b }</code>

20. Practice roadmap

Week 1 — Syntax ki aadat 🍽️

- Part 1 padho, har example playground mein chalao
- Easy problems: Two Sum, Reverse String, Valid Palindrome, Contains Duplicate, Fizz Buzz, Best Time to Buy and Sell Stock
- Focus: `Vec`, loops, `as` casting, `HashMap` entry pattern

Week 2 — Ownership se dosti 🤝

- Part 2 dobara padho
- Har function mein socho: `&`, `&mut`, ya owned?
- Problems: Valid Anagram, Group Anagrams, Valid Parentheses, Merge Intervals, Longest Substring Without Repeating Characters

Week 3 — Patterns 🍀

- Binary search, sliding window, two pointers, stack, heap
- Problems: Search in Rotated Sorted Array, Koko Eating Bananas, Daily Temperatures, Kth Largest Element, Top K Frequent

Week 4 — Graph + Recursion 🌳

- Number of Islands, Course Schedule (topo), Network Delay Time (Dijkstra), Subsets, Permutations, Combination Sum
- Tree: Max Depth, Invert Tree, Level Order, Validate BST

Week 5+ — DP & mixed 🚀

- Climbing Stairs, House Robber, Coin Change, LCS, Edit Distance, Longest Increasing Subsequence
- Linked list: Reverse, Merge Two Sorted Lists

Last advice 🙏

1. **Pehle 2 hafte compiler se ladai hogi.** Normal hai. Har error ek free lesson hai.
2. **Error message poora padho** — Rust ka compiler aksar `help: consider ...` likh ke solution de deta hai.
3. **Doubt ho toh `.clone()` maaro**, solution chalao, phir optimize karo. Pehle working, phir perfect.
4. **Indices > pointers.** Graph, tree, trie — sab kuch `Vec` + `usize` index se banao jab bhi possible ho.
5. **`i64` default rakho** values ke liye, `usize` indices ke liye. 90% type errors gayab.
6. `cargo clippy` chalao — ye tumhe idiomatic Rust sikhayega.

Chalo, ab jao aur pehla problem solve karo. Rust ke saath tumhari dosti pakki hogi. 🦀💪